APPENDIX C

Quick Reference

This appendix collects the book's answers in the form you need them while writing code. Each row points to the chapter that explains it.

C.1 I want to...

I want to…	Write	Chapter
return a reference derived from the only reference input	elide: `fn f(x: &T) -> &U`	3
return a reference derived from one of several inputs	name it: `fn f<'a>(x: &'a T, y: &U) -> &'a V`	3
say that the result may come from either input	`fn f<'a>(x: &'a T, y: &'a T) -> &'a T`	1, 3
store a reference in a struct	`struct S<'a> { r: &'a T }`	2
return data from a struct's referent, not from the `&mut self` borrow	`impl<'a> S<'a> { fn f(&mut self) -> &'a T }`	2
make a hidden lifetime visible	`Iter<'_, T>`, `Formatter<'_>`	6
implement a trait for a borrowing type	`impl Trait for Wrapper<'_>`	6
return an iterator over `&self`	`-> impl Iterator<Item = &T>` (captures `self` in edition 2024)	8
return `impl Trait` that must not borrow some argument	`-> impl Trait + use<'a>`	8
return a boxed closure or trait object that borrows an argument	`Box<dyn Fn(..) -> .. + '_>`	6, 8
store a trait object in a long-lived struct	`Box<dyn Trait>` (`'static`) or `struct S<'a> { b: Box<dyn Trait + 'a> }`	8
return an error that borrows its input	`Box<dyn Error + '_>`, or own the data in the error	8
run a closure on another thread with borrowed data	`thread::scope(|s| { s.spawn(|| ..); })`	7
make generic code spawnable	`T: Send + 'static`	7
accept a callback for borrows the function itself creates	`F: Fn(&T) -> U` (that is, `for<'a> Fn(&'a T) -> U`)	9
relate a callback's result to one of two arguments	`F: for<'a> Fn(&'a T, &U) -> &'a V`	9
accept an async callback whose future borrows the argument	`F: AsyncFn(&T) -> U`	9, 11
iterate an owned generic collection by reference	`where for<'a> &'a C: IntoIterator<Item = &'a T>`	9
accept only types that do not borrow from the input	`T: for<'de> Trait<'de>` (serde: `DeserializeOwned`)	9
yield items that borrow from the iterator itself	a GAT: `type Item<'a> where Self: 'a;`	10
hand out a reference into a `RefCell`	`Ref::map(cell.borrow(), |v| &v.field)`	12
tie a raw-pointer wrapper to a borrow	`PhantomData<&'a T>`	13
hold text that is usually borrowed, sometimes owned	`Cow<'a, str>`	14
share immutable text without a lifetime parameter	`Arc<str>` / `Rc<str>`	14

C.2 What the compiler writes for you

You write	The compiler reads	Chapter
`fn f(x: &T) -> &U`	`fn f<'a>(x: &'a T) -> &'a U`	3
`fn m(&self, k: &K) -> &V`	the output borrows from `self`	3
`fn f(x: &T, y: &T) -> &U`	error E0106: nothing to choose from	3
`static S: &str`, `const C: &str`	`&'static str`	3
`impl S { const C: &str = ...; }`	`&'static str` (also in `impl`s and traits with lifetime parameters since 1.99)	3
`&'a T` in a signature	also the bound `T: 'a`	1
`fn f(x: &str) -> impl Trait` (edition 2024)	the opaque type captures the lifetime of `x`	8
`Box<dyn Trait>`	`Box<dyn Trait + 'static>`	8
`&'a dyn Trait`	`&'a (dyn Trait + 'a)`	8
`F: Fn(&T) -> &U`	`F: for<'a> Fn(&'a T) -> &'a U`	9
`fn(&T) -> &U`	`for<'a> fn(&'a T) -> &'a U`	9

C.3 Rules of thumb

1. A lifetime is a region of program points, not a duration. Annotations describe relationships between borrows; they never make anything live longer.
2. Only signatures need lifetimes; inside a body the compiler infers them.
3. The signature is the contract: the body is checked against it, and callers are checked against it and nothing else.
4. Elision uses the lifetime of the `self` reference if there is one, else the lifetime of the only parameter that has exactly one, else fails. It never looks at the body.
5. The `&mut self` borrow and a struct's own lifetime parameter are different lifetimes; return the one you mean.
6. `'_` means "a lifetime I do not need to name", never "the same as the other one".
7. `+ '_` lets a trait object borrow. An `impl Trait` return type captures every lifetime in scope (edition 2024); `use<..>` narrows it.
8. Shared references are covariant; `&mut T` and `Cell<T>` are invariant in `T`.
9. `&'static T` is a reference to data that lives until the end of the program; `T: 'static` is a bound that every owned type satisfies.
10. `Box<dyn Trait>` means `+ 'static`; `&'a dyn Trait` means `+ 'a`.
11. A lifetime parameter is picked by the caller and outlives the whole body. When the function itself lends data out, the bound must be higher-ranked (`for<'a>`, or the elided `Fn(&T)` form).
12. When tempted to add `'static`, first try owning, scoping, or `Cow`.
13. `&'a mut T<'a>` borrows a value for the rest of its life. Use two lifetimes.